49

BÖLÜM: FLUXCD ILE MODERN GITOPS – TEK GERÇEK KAYNAK GIT! | MODULE: MODERN GITOPS WITH FLUXCD – GIT AS THE SINGLE SOURCE OF TRUTH

GitOps Nedir? Neden `kubectl apply` Artık Yasak?

What is GitOps? Why Manual `kubectl apply` is an Anti-Pattern

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Bir şirkette 10 farklı yazılımcının canlı sunucuya bağlanıp kafasına göre `kubectl apply` yapması, bir restorandaki 10 aşçının aynı çorbaya gizlice farklı tuzlar atmasına benzer (Sonuç tam bir zehirlenme faciası!). Kim ne değiştirdi, dün ne çalışıyordu kimse bilmez! GitOps'ta ise kural kesindir: Canlı sunucuya KİMSE el süremez! Tek gerçek kaynak Git deposudur. Bir şey değişecekse Git'e commit atılır, kümenin içindeki bekçi robot (Flux) değişikliği çeker!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Ten engineers SSH'ing into production to fire off manual `kubectl apply` commands is like ten rogue chefs secretly dumping different spices into the soup without telling each other (Total disaster!). Nobody knows who broke production! GitOps establishes an unbreakable law: Git is the Single Source of Truth. Nobody has cluster write access. You commit changes to Git, and a dedicated cluster agent (Flux) pulls and reconciles them!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Geleneksel CI/CD sistemleri 'Push' (İtme) modeliyle çalışır: Jenkins veya GitLab sunucusu doğrudan K8s üretim kümesinin `cluster-admin` şifresine sahiptir ve dışarıdan kümeye bağlanıp değişiklik basar (Büyük güvenlik riski!). GitOps ise 'Pull' (Çekme) modeliyle çalışır: K8s kümesinin dışarıya hiçbir portu veya yönetim yetkisi açılmaz. Küme içinde çalışan Flux denetleyicisi, dışarıdaki Git deposunu periyodik olarak yoklar (poll) ve kümedeki 'yapılandırma sapmalarını' (configuration drift) otomatik onarır.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Traditional CI/CD relies on an insecure 'Push' model: external build runners possess broad `cluster-admin` credentials to inject updates into production. GitOps inverts this via a zero-trust 'Pull' architecture: the cluster exposes no inbound management ports. Instead, an in-cluster agent (Flux) continuously pulls from Git, comparing Git's declared state against live cluster reality and reversing any unauthorized configuration drift.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# Geleneksel Tehlikeli Yöntem (Mühendis terminalinden canlıya müdahale):
kubectl apply -f deployment.yaml # YASAK! İz bırakmaz, geri alınamaz!

# GitOps Yolu (Güvenli, Denetlenebilir, Kurumsal Standart):
git add clusters/production/coffee-app/
git commit -m "feat(coffee): upgrade spring-boot to v1.2.0"
git push origin main

# Küme içindeki Flux robotu değişikliği saniyeler içinde otomatik çeker!
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

GitOps kurulu bir kümede bir mühendis gidip canlıda `kubectl edit deployment` ile değişiklik yaparsa ne olur? Flux bir dakika içinde bu değişikliği yakalar, 'Git deposunda böyle bir şey yazmıyor!' diyerek değişikliği ezer ve sistemi Git'teki haline geri döndürür!

🇬🇧 WATCH OUT

If a rogue engineer modifies a live pod via `kubectl edit`, Flux detects the drift within 60 seconds, nukes the manual edit, and enforces the exact state declared in Git.

🎯 Özet Çıkarımlar

- Git tüm altyapının ve yazılımların tek resmi versiyon kütüğüdür.
- Çekme (Pull) modeli sayesinde küme erişim anahtarları dışarıya sızmaz.
- Tüm dağıtım geçmişi `git log` üzerinden kimin ne yaptığıyla açıkça izlenir.

🎯 Key Takeaways

- Git is the immutable, auditable single source of truth for the entire cluster.
- The Pull model eliminates the security hazard of storing cluster tokens in external CI.
- Complete auditability and compliance: every cluster change corresponds to a signed commit.